LEARN BEING FORWARD

Programming — Java Programming

Exception handling & I/O

COMPLETE LEARNING GUIDE

What You'll Learn

Real programs fail in predictable ways — a file might not exist, a network call might time out, a user might type text where a number was expected — and Java's exception-handling model forces you to decide, at compile time for checked exceptions, how each of those failures gets handled rather than letting them crash the program unceremoniously. This sub-module covers the checked-vs-unchecked exception split and why it exists, try/catch/finally mechanics including the much safer try-with-resources syntax, writing custom exceptions, preserving the original cause when wrapping one exception in another, and modern file I/O with java.nio.file alongside the classic Scanner and buffered-stream APIs — closing with the resource-management discipline that separates production-quality Java from code that merely happens to work in a demo.

1. Checked vs Unchecked Exceptions

Every exception in Java descends from Throwable, which splits into Error (serious JVM-level problems like OutOfMemoryError that applications generally shouldn't try to catch) and Exception. Exception itself splits further: checked exceptions (any subclass of Exception that isn't a RuntimeException, like IOException) must be either caught or declared in a method's throws clause — the compiler enforces this — because they represent recoverable conditions external to the program's own logic, like a missing file. Unchecked exceptions (RuntimeException and its subclasses, like NullPointerException or IllegalArgumentException) represent programming errors and are not compiler-enforced, since requiring every method to declare every possible NullPointerException would make the type system unusable.

- Checked exceptions model recoverable, expected failure conditions (file not found, network timeout) — the compiler forces you to acknowledge them.
- Unchecked exceptions (RuntimeException and subclasses) typically signal a bug in the code itself, not an expected external condition.

2. try/catch/finally and try-with-resources

Code that might throw goes in a try block; one or more catch blocks handle specific exception types (ordered from most specific to most general, since Java checks them top-to-bottom); an optional finally block always executes, whether or not an exception occurred, making it the traditional place for cleanup. Since Java 7, try-with-resources automatically closes any resource implementing AutoCloseable at the end of the block — in the reverse order resources were opened — eliminating the manual finally-block close() calls that were historically a huge source of resource leaks and nested-try ugliness. Multi-catch (catch (IOException | SQLException e))

handles several exception types identically in one block without duplicating code.

```java
import java.io.*;

public void readFirstLine(String path) {
    try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
        System.out.println(reader.readLine());
    } catch (FileNotFoundException e) {
        System.err.println("File not found: " + path);
    } catch (IOException e) {
        System.err.println("Error reading file: " + e.getMessage());
    }
    // reader.close() is called automatically, even if an exception was thrown
}
```

3. Creating and Using Custom Exceptions

Defining your own exception class — by extending Exception for a checked exception or RuntimeException for an unchecked one — lets you represent domain-specific failure conditions (InsufficientFundsException, InvalidOrderException) far more clearly than throwing a generic RuntimeException with a string message. The custom class should call one of the parent's constructors via super(...) to preserve the message (and optionally the cause) that the rest of the exception-handling machinery relies on. The choice between checked and unchecked for your own exception follows the same logic as the built-in ones: checked if callers can reasonably be expected to recover, unchecked if it represents a programming mistake they should fix rather than handle.

```java
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

public void withdraw(double amount, double balance) throws
InsufficientFundsException {
    if (amount > balance) {
        throw new InsufficientFundsException("Requested " + amount + " exceeds
balance " + balance);
    }
}
```

4. Exception Chaining and Preserving the Cause

When catching a low-level exception and throwing a more meaningful higher-level one, passing the original exception as the cause argument (new ServiceException("Failed to load config", originalException)) preserves the full original stack trace inside the new exception's getCause(). Discarding the original exception and throwing a brand-new one with only a string message destroys the debugging trail that points to exactly where and why the failure actually originated — in production code this difference can turn a five-minute fix into an hours-long investigation.

```java
try {
    loadConfigFile();
} catch (IOException e) {
    throw new RuntimeException("Could not start application", e); // cause preserved
}
```

5. Modern File I/O with java.nio.file

The NIO.2 API (java.nio.file), introduced in Java 7, is the preferred way to work with files in new code, favored over the legacy java.io.File class for its clearer exceptions and richer feature set. A Path represents a file or directory location (platform-independent, built with Path.of(...) or Paths.get(...)), and the Files utility class provides static methods like Files.readString, Files.writeString, Files.exists, and Files.createDirectories that cover the vast majority of everyday file operations in one line each, without manually managing streams for simple cases.

```java
import java.nio.file.*;
import java.io.IOException;

Path logPath = Path.of("logs", "output.txt");

try {
    Files.writeString(logPath, "Application started\n",
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    String contents = Files.readString(logPath);
    System.out.println(contents);
} catch (IOException e) {
    System.err.println("File operation failed: " + e.getMessage());
}
```

6. Buffered Streams and Scanner for Text Processing

Raw byte streams (InputStream/OutputStream) and character streams (Reader/Writer) perform a system call for every single read or write, which is slow when done one byte or character at a time. Wrapping them in BufferedReader/BufferedWriter batches reads and writes into larger chunks internally, dramatically reducing the number of actual I/O operations for line-by-line or chunked text processing. Scanner, by contrast, is built for tokenizing input into typed values (nextInt(), nextDouble(), next() for a whitespace-delimited word) and is the standard tool for reading structured console input, though it's not the fastest choice for parsing huge files line-by-line — that's BufferedReader's job.

```java
import java.util.Scanner;

Scanner scanner = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = scanner.nextInt(); // throws InputMismatchException on non-numeric input
System.out.println("You are " + age + " years old.");
```

7. Best Practices: Resource Management and Not Swallowing Exceptions

Every resource that implements AutoCloseable — files, sockets, database connections — should be opened inside a try-with-resources statement so it's guaranteed to close even if an exception is thrown mid-operation; manual finally-block closing is still legal but far more error-prone (the close() call itself can throw, and nested resources multiply the boilerplate quickly). An empty catch block — catch (Exception e) {} — is one of the most damaging anti-patterns in Java code, because it silently discards evidence of a real failure; at minimum, log the exception, and prefer catching the most specific exception type the code can actually recover from rather than a blanket Exception or Throwable.

Common Pitfalls

- Catching the broad Exception (or even Throwable) type instead of the specific exception that can actually occur, hiding unrelated bugs.
- Forgetting that checked exceptions must be caught or declared with throws in the method signature — the compiler enforces this for Exception subclasses that aren't RuntimeException.
- Manually closing resources in a finally block instead of using try-with-resources, risking a leaked resource if an earlier line throws before the close() call.
- Throwing a brand-new exception from inside a catch block without passing the original exception as the cause, losing the real stack trace.
- Using Files.readAllLines or readString on an enormous file, loading the entire contents into memory at once instead of streaming it.
- Leaving an empty catch block that silently swallows an exception instead of logging it or handling it meaningfully.
- Mixing up Scanner's nextInt()/nextLine() calls, since nextInt() leaves the trailing newline in the buffer for the next nextLine() to pick up unexpectedly.

Key Takeaways

- Checked exceptions (compiler-enforced) model recoverable external conditions; unchecked exceptions (RuntimeException) model programming errors.
- try-with-resources auto-closes any AutoCloseable resource, even on an exception, and should be preferred over manual finally-block cleanup.

- Always pass the original exception as the cause when wrapping it in a new one, to preserve the real stack trace.
- java.nio.file (Path + Files) is the modern, preferred file I/O API over the legacy java.io.File for most new code.
- BufferedReader/BufferedWriter reduce the number of actual I/O system calls; Scanner is for tokenizing typed input, not raw performance.
- Never leave a catch block empty — at minimum, log the exception so failures aren't silently discarded.

Learn More

Oracle — The Java Tutorials: Exceptions
https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html

Oracle — The Java Tutorials: Basic I/O (NIO.2)
https://docs.oracle.com/javase/tutorial/essential/io/index.html

Baeldung — Java try-with-resources
https://www.baeldung.com/java-try-with-resources